Eureka Server is a **service registry** used in microservices architecture, mainly with **Spring Cloud Netflix Eureka**. It helps different services discover and communicate with each other dynamically without hardcoding URLs.

🔷 Why Do We Need Eureka Server?

In a microservices system:

- You may have multiple services (User Service, Order Service, Payment Service, etc.)
- Each service can run on different ports or even different machines
- Services can scale up/down dynamically

👉 Problem:
How does one service know **where another service is running**?

👉 Solution:
Use **Eureka Server** as a central registry.

🔷 What Does Eureka Server Do?

Eureka Server acts like a **phone directory** for services.

It:

1. **Registers services** (when they start)
2. **Stores their locations (IP + Port)**
3. **Helps other services discover them**
4. **Removes unavailable services automatically**

🔷 How It Works (Step-by-Step)

1. **Service Starts**
    - Example: User-Service starts
    - It registers itself with Eureka Server
2. **Registration**
    - Eureka stores:

User-Service → localhost:8081

3. **Another Service Needs It**
    - Example: Order-Service wants to call User-Service
4. **Discovery**

- Instead of hardcoding URL, it asks Eureka:

Where is User-Service?

5. **Eureka Responds**
    - Returns: localhost:8081
6. **Communication**
    - Order-Service calls User-Service using that info

🔷 Key Concepts

1. Service Registration

Each microservice registers itself with Eureka.

2. Service Discovery

Services query Eureka to find other services.

3. Heartbeat Mechanism

- Services send heartbeat every ~30 seconds
- If missed → Eureka removes the service

🔷 Types of Eureka Components

1. Eureka Server

- Central registry
- Keeps track of all services

2. Eureka Client

- Any microservice that:
    - Registers itself
    - Fetches other services

🔷 Advantages

✓ No hardcoded URLs
✓ Dynamic scaling support
✓ Fault tolerance
✓ Load balancing (with Ribbon / Spring Cloud LoadBalancer)
✓ Easy microservice communication

🔷 Simple Analogy

Think of **Eureka Server as a Zomato/Swiggy app**:

- Restaurants = Microservices
- App = Eureka Server
- You = Service requesting another service

Instead of remembering restaurant addresses, you just search on the app.

🔷 Example Configuration (Spring Boot)

Enable Eureka Server

```
@EnableEurekaServer
@SpringBootApplication
public class EurekaServerApplication {
   public static void main(String[] args) {
      SpringApplication.run(EurekaServerApplication.class, args);
   }
}
```

Client Configuration

```
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

🔷 Default URL

Eureka Dashboard:

http://localhost:8761

🔷 Interview Tip 💡

👉 If asked:
"Why use Eureka instead of hardcoding URLs?"

Answer:

Because in microservices, instances change dynamically. Eureka provides service discovery, making the system scalable and loosely coupled.

🔷 Part 1: Step-by-Step — Integrate Eureka with Spring Boot

We'll build:

- 1 Eureka Server
- 2 Microservices (User-Service, Order-Service)

✅ Step 1: Create Eureka Server

1. Add Dependency (Spring Initializr)

Add:

- Spring Web
- Eureka Server

2. Enable Eureka Server

```
@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
   public static void main(String[] args) {
      SpringApplication.run(EurekaServerApplication.class, args);
   }
}
```

3. application.properties

```
server.port=8761

eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false
```

👉 Start the app and open:

http://localhost:8761

✅ Step 2: Create a Microservice (User-Service)

1. Add Dependency

- Spring Web
- Eureka Discovery Client

2. Enable Eureka Client

```
@SpringBootApplication
@EnableEurekaClient
public class UserServiceApplication {
   public static void main(String[] args) {
```

```
        SpringApplication.run(UserServiceApplication.class, args);
    }
}
```

3. application.properties

```
spring.application.name=USER-SERVICE
server.port=8081

eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

✅ **Step 3: Create Another Microservice (Order-Service)**

Same steps, just change name & port:

```
spring.application.name=ORDER-SERVICE
server.port=8082

eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

✅ **Step 4: Run Everything**

Start in order:

1. Eureka Server
2. User-Service
3. Order-Service

👉 Go to dashboard:

http://localhost:8761

You'll see:

```
USER-SERVICE
ORDER-SERVICE
```

✅ **Step 5: Inter-Service Communication (IMPORTANT)**

Instead of:

http://localhost:8081/users

Use:

http://USER-SERVICE/users

🔷 **Using RestTemplate (with Load Balancing)**

```
@Bean
@LoadBalanced
public RestTemplate restTemplate() {
   return new RestTemplate();
}
```

Call service:

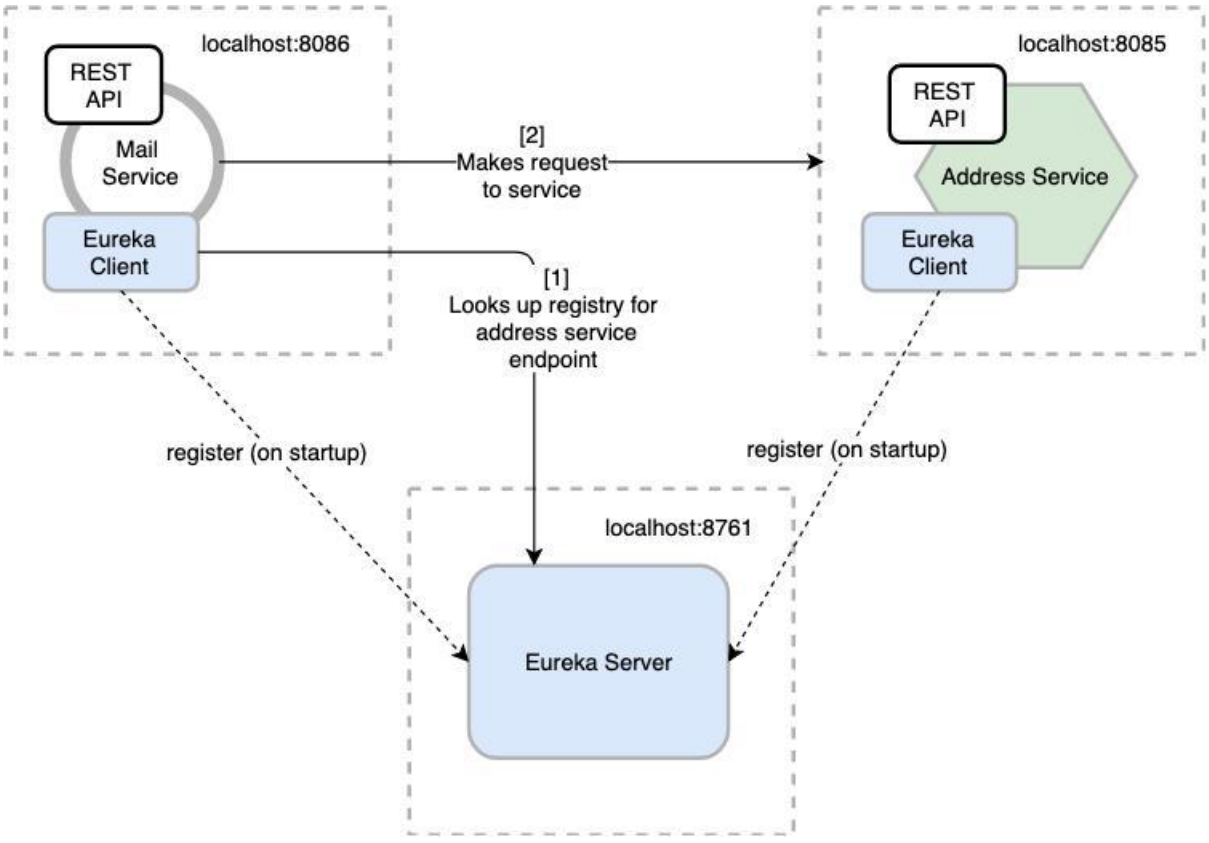
```
String response = restTemplate.getForObject(
   "http://USER-SERVICE/users", String.class);
```

🔷 **Using Feign Client (Recommended in Interviews)**

```
@FeignClient(name = "USER-SERVICE")
public interface UserClient {

   @GetMapping("/users")
   List<String> getUsers();
}
```

🔷 Part 2: Real-World Architecture Diagram



🔥 **Interview Tips (Very Important)**

💡 **Common Questions:**

👉 **Q: What happens if Eureka Server goes down?**

- Clients still work using cached registry (self-preservation mode)

👉 **Q: Difference between Eureka Client & Discovery Client?**

- DiscoveryClient is generic
- EurekaClient is Netflix-specific

👉 **Q: Why not hardcode URLs?**

- Services scale dynamically
- Instances change frequently

👉 **Q: What replaces Eureka in modern systems?**

- Kubernetes Service Discovery
- Consul

🚀 **Pro**